

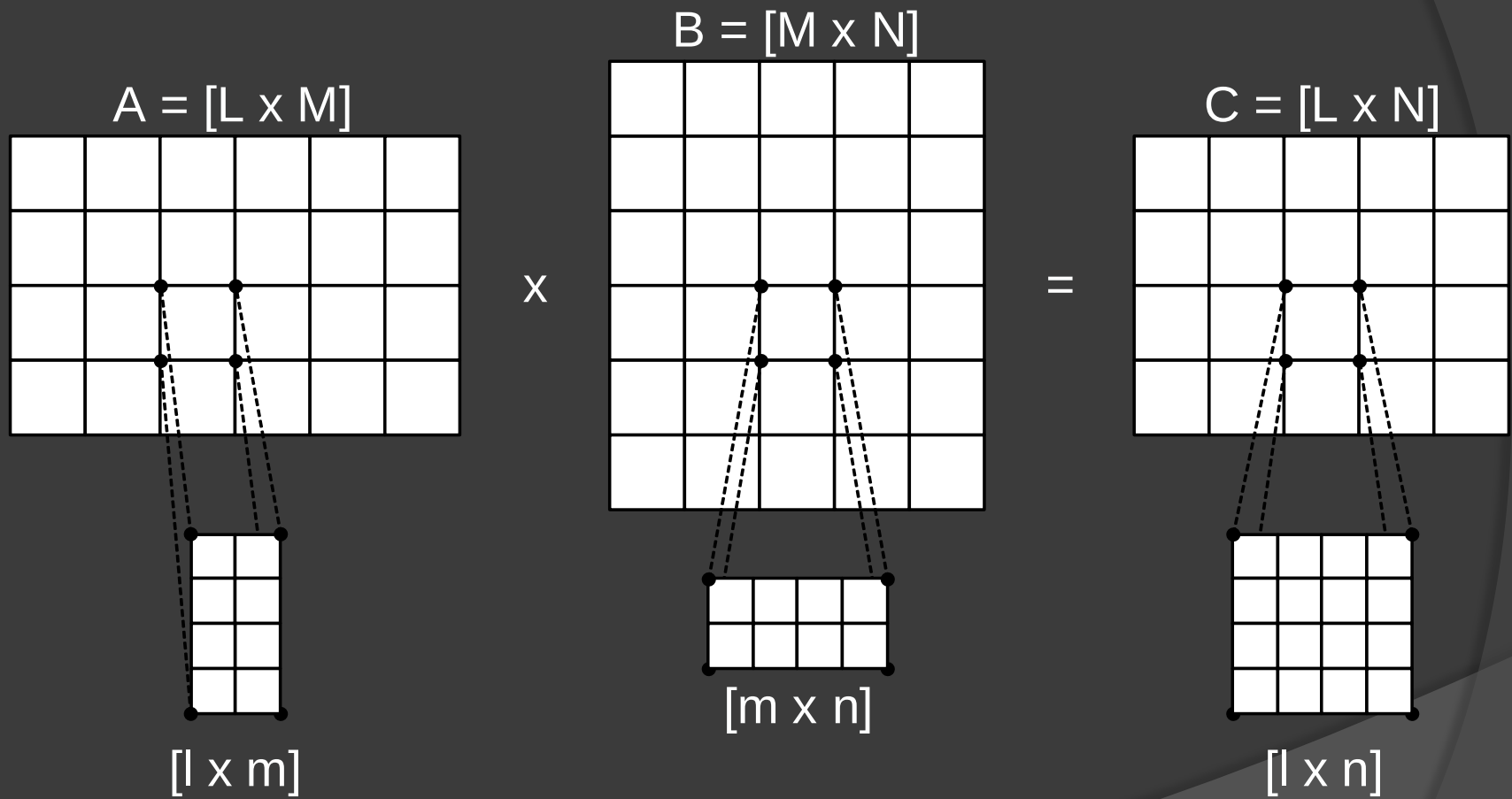
# **ЛАБОРАТОРНАЯ №1**

## **«ВЕКТОРИЗАЦИЯ»**

# Задание

- ⦿ перемножить 2 матрицы следующими способами:
  - результат – матрица C1:
    - с включенной векторизацией
    - с выключенной векторизацией
  - результат – матрица C2:
    - ручная векторизация с использованием SSE2-инструкций (либо новее)
      - ассемблер
      - intrinsics (альтернатива ассемблеру)
- ⦿ элементы матрицы
  - своя матрица меньшего размера (см. сл.слайд)
- ⦿ размер внешней матрицы подбирается самостоятельно
  - время получения матрица C1 от нескольких секунд
- ⦿ размер матрицы / тип входных данных:
  - **определяется преподавателем**

# Задание



# Задание

## ● Обязательные условия:

- в одном проекте минимум (!) 2 функции:
  - с автоматической векторизацией
  - с ручной векторизацией
- результат матриц C1 и C2 должны **ПОЛНОСТЬЮ** совпадать
  - никакого вывода фрагментов матрицы НЕ делать
- время работы SSE2-версии *не* медленнее версии с автоматической векторизацией

# Замечания по векторизации

- ⦿ Зависит от среды разработки:
  - Visual Studio 2012 и новее
  - GCC (версия неизвестна, точно с 2009 года)
    - ПРОБЛЕМА: как отключить векторизацию ???
- ⦿ способы доказать:
  - дизассемблер
    - спорный способ есть на MSDN
    - принимается в определенных условиях 😊
  - Intel vTune
- ⦿ Ускорение векторизованной версии:
  - float: ~3-3,5 раза
  - double: ~1,2-1,5 раза

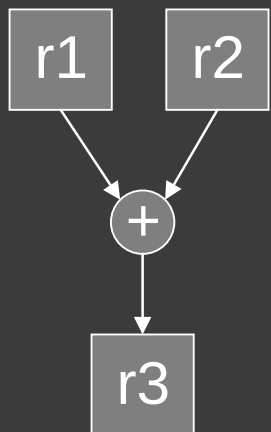
# Способы заработать бонусы

- ◎ универсальная версия с произвольными размерами матрицы
- ◎ доказательство векторизации с помощью vTune
- ◎ выявить ВСЕ «узкие» места производительности (аппаратная часть) с помощью vTune
- ◎ качественное, УНИКАЛЬНОЕ (!) и интересное решение (*на усмотрение преподавателя*)

# Векторизация

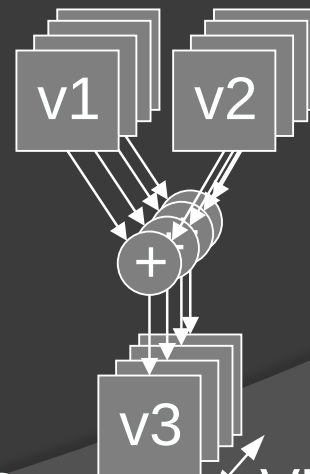
- выполнение одной инструкции над вектором данных
- в лаб.работе **транспонировать матрицы запрещено**

Скалярная  
операция



`add.d r3, r1, r2`

Векторная  
операция



`addvec.d v3, v1, v2`

VL — длина  
вектора

# Векторизация

## ⦿ типы данных:

- int (не во всех SIMD-наборах)
- double, float (кроме MMX)

## ⦿ наборы SIMD-инструкции:

- MMX (Multi-media extension, 1997 г.)
  - 64 бит, регистры mmx
- SSE (Streaming SIMD Extension, 1999 г.)
  - 128 бит, регистры xmm
- AVX (Advanced Vector Extension, 2011 г.)
  - 256 бит, регистры ymm
- AVX-512 (2016 г.)
  - 512 бит, регистры zmm

# Требования к коду

- ⦿ итерации цикла должны быть независимыми
- ⦿ код в цикле «просчитываемый»
- ⦿ нарушение потока команд
- ⦿ внутренний цикл

# Рекомендуемое чтение

- ◎ [24504501.pdf](#) – то, как рекомендуется перемножать матрицы от Intel
  - P.S. Версия для тех, кому скучно ☺